



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #1
File Handling in C***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: File handling in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Data created during program execution is stored temporarily in variables. However, once the program ends, this data is lost. To preserve data even after the program terminates, **files** are used. File handling allows a C program to **create, read, write, and modify files** stored on disk. This is crucial for tasks such as **saving user input, processing large datasets, logging system activities, or reading configuration files.**

File handling enables the below tasks:

- **Persistent data storage** beyond the life of a program.
- **Data sharing** between programs.
- **Handling large volumes of data** efficiently (which might not fit in memory)

File handling allows programs to **create, open, read, write, append, and close files** stored on the computer's storage device. File handling gives us the tools(functions) to manage the different types of files efficiently.

Real-Life Examples

1. Saving User Data in a Program
2. Creating Logs for a System - Record program events, errors, or status updates
3. Storing Game Scores - Save high scores or game progress that can be retrieved later.
4. Reading Configuration Settings - Load program preferences from a file during startup.
5. Writing Results from a Scientific Simulation
6. Handling Financial Transactions - Record billing or transaction data securely in a file.
7. Maintaining Audit Trails - Keep track of user actions and system access for security reviews.

File Classification

Files are referred to as data files in C. There are basically two distinct types of data files available in C.

- Text Files
- Binary Files

Text Files

The simplest type of file is a **text file**, typically created with a **.txt extension** using any basic text editor. Internally, **it stores data as ASCII characters**, making the content **easily readable by humans**. However, text files are not secure, as the information is stored in plain text, and they tend to occupy more storage space compared to binary files.

Binary Files

A binary file stores data using the binary number system (0s and 1s) and typically has a **.bin extension**. Since it saves information in the same format as it's stored in computer memory, it uses **less storage space**. Unlike text files, binary files are **not human-readable**, making them a safer option for **storing sensitive or structured data**.

Major Operations on Files

- Creation of a new file or Opening an existing file
- Reading data from a file
- Writing data in a file
- Closing a file
- Rewinding a File Pointer
- Moving the File Pointer
- Getting the Current File Position
- Checking for End-of-File
- Removing a File
- Renaming a File

To perform any operation on Files, the **physical filename**, the **logical filename** and the **mode** must be connected using `fopen()`

- **Physical Name:** A file is maintained by the OS. The OS decides the naming convention of a file.
- **Logical Name:** In a C Program, identifier is used to refer to a file. Also called as FileHandle.
- **Mode:** Can be read only, write only, append or a combination of these.

1. Creation of a new file or Opening an existing file

To create a new file, one has to use a **built-in FILE structure**. Create a pointer to FILE type. This must hold a logical reference to physical file on disk (hard disk) using the `fopen()` function.

```
FILE *fp;  
fp = fopen("filepath", "mode");
```

fopen(): Opens a file in the specified mode and **returns a FILE pointer — a reference to a structure that holds information about the file's attributes**. If the file is opened successfully, this pointer is returned; otherwise, NULL is returned. **File opening can fail for various reasons, such as the file not existing or lacking the necessary permissions for the requested mode**. The first argument, `filePath`, is a pointer to a string representing the name of the file to be opened. The second argument, `mode`, defines the type of access (read, write, append, etc.). It's important to note that initializing a FILE pointer does not load the entire file into memory — additional file handling functions must be used to access or modify the file's content.

A file in C can be opened using **different modes based on the desired operation**.

- **"r"** – **Opens an existing file for reading only**. The file must already be present; otherwise, the operation fails.
- **"w"** – **Opens a file for only writing**. If the file exists, its contents are erased. If it doesn't, a new empty file is created.
- **"a"** – **Opens a file to append data at the end**. If the file does not exist, it is created automatically.
- **"r+"** – **Opens an existing file for both reading and writing**. The file must exist; otherwise, the operation fails.
- **"w+"** – **Creates a new file for both reading and writing**. If the file exists, it is overwritten.
- **"a+"** – **Opens a file for both reading and appending**. Data is always added to the end of the file. If the file doesn't exist, it is created.

fclose(): When a file is opened, certain data structures are created in memory to manage the file operations. Once the file is no longer needed, these **resources should be released by closing the file using the fclose() function**. This action **closes the stream, flushes any remaining data in the buffers to the file, and frees associated memory**. If the file is closed **successfully, fclose() returns 0; otherwise, it returns EOF to indicate an error**.

Consider, **data.txt** contains the below data.

welcome to file
handling
in C

Coding Example_1: Demo of fopen and fclose.

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r"); //file must be existing to open in read mode. Else returns NULL
    if(fp== NULL)
        printf("file cannot be opened\n");
    else {
        //perform reading here in else using any functions .then close the file
        printf("File can be accessed for reading\n");
        fclose(fp);
    }
    return 0;
}
```

```
C:\Users\Dell>a
File can be accessed for reading
```

2. Read/Write data from/to a file

This can be categorized into the following types:

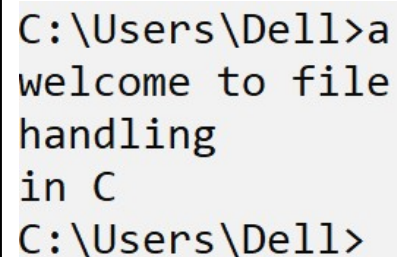
- **Character-based I/O** – Reading and writing one character at a time.
- **String-based I/O** – Handling entire lines or strings at once.
- **Formatted I/O** – Input and output using specific formats, similar to scanf() and printf().
- **Block I/O** – Reading from or writing to files in blocks of data, typically used for binary files.

Character based functions:

- **fputc(int ch, File_pointer):** Writes a character (an unsigned char) specified by the argument ch to the specified stream and advances the position indicator for the stream. If there are no errors, the same character that has been written is returned. If an error occurs, EOF is returned and the error indicator is set.
- **fgetc(file_pointer):** Reads a character from the file and returns the character read as an unsigned char cast to an int(ascii) and advances the file pointer to the next byte to be read or EOF on end of file **or error**.
- **getc(FILE *fp) and putc(char ch, FILE *fp):** The operation performed by getc() and putc() functions is the same as that performed by fgetc() and fputc() functions.

Coding Example_2: Demo of fgetc and fputc

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r"); // file must be existing
    if(fp== NULL)
        printf("file cannot be opened\n");
    else
    {
        char ch;
        while((ch=fgetc(fp)) != EOF)
        {
            //printf("character read is %c",ch);
            //printf("%c",ch);
            fputc(ch,stdout);// writing to standard output that is terminal
        }
        fclose(fp);
    }
    return 0;
}
```



String based Functions:

- **fputs(const char *str, int n, FILE *fp):** Writes a string to the specified stream up to but not including the null character. Returns a non-negative value, or else on error it returns EOF.

- **fgets(char *str, FILE *fp):** Reads a line from the specified stream and stores it into the string pointed to by str. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first. On success, the function returns the same str. On error or if EOF is encountered, it returns NULL.

Coding Example_3: Demo of fgets and fputs. out.txt created with the contents of data.txt

```
#include<stdio.h>
```

```
int main(){
```

```
    FILE *fp1 = fopen("data.txt","r");
```

```
    FILE *fp2 = fopen("out.txt","w");
```

```
    char ch[500];
```

```
    if(fp1==NULL || fp2 ==NULL)
```

```
        printf("issue in opening the file\n");
```

```
    else {
```

```
        while(fgets(ch,500,fp1) != NULL) // change 500 to 2 and check the output
```

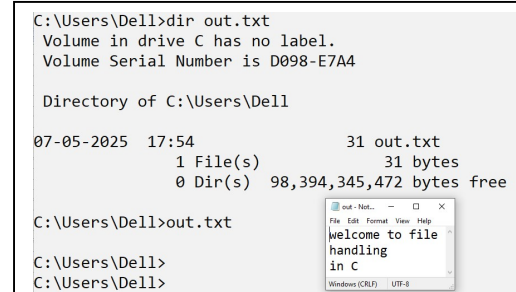
```
        {      fputs(ch, fp2);      }
```

```
        fclose(fp1); fclose(fp2);
```

```
    }
```

```
    return 0;
```

```
}
```



```
C:\Users\Dell>dir out.txt
Volume in drive C has no label.
Volume Serial Number is D098-E7A4

Directory of C:\Users\Dell

07-05-2025  17:54                31 out.txt
               1 File(s)                31 bytes
               0 Dir(s)  98,394,345,472 bytes free

C:\Users\Dell>out.txt
C:\Users\Dell>
```

Formatted IO Functions:

When a data file is organized in a specific format—such as a line containing five integers separated by spaces—and the programmer knows this format in advance, it is more efficient to use formatted file I/O functions instead of reading character by character. These functions allow structured reading and writing based on expected data patterns. The `fscanf()` function works like `scanf()`, but it takes a file pointer as its first argument to read data directly from a file. `fprintf()` is similar to `printf()`.

- **fprintf(file_pointer,"format_string", arg1,arg2,...):** Sends formatted output to a stream. If successful, the total number of characters written is returned otherwise, a negative number is returned .
- **fscanf(file_pointer,"format_string",&arg1,&arg2,...):** Reads formatted input from a stream and returns the number of input items successfully matched and assigned, which can be fewer than provided for, or even zero in the event of an early matching failure.

Assume, Formatted.txt file contains the below data.

```
10    20    30
40    10    20
```

Coding Example_4: Demo of fscanf and fprintf to read an integer from a file and write to the terminal

```
#include<stdio.h>
int main(){
    FILE *fp1 = fopen("Formatted_data.txt","r");
    if(fp1==NULL)
        printf("issue in opening the file\n");
    else{
        int a;          fscanf(fp1,"%d",&a);
        fprintf(stdout,"%d",a); fclose(fp1);
    }
    return 0;
}
```

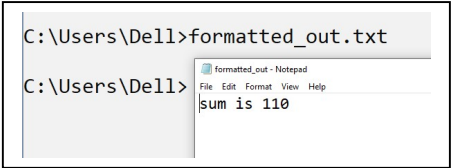


```
C:\Users\De11>a
10
```

Coding Example_5: Reading a set of integers from a file called numbers.txt and writing the sum of these integers to a new file.

Version 1: We know the number of integers to be read from the data file – n is hardcoded.

```
void read(int *a, int n,FILE *fp);
int find_sum(int *a, int n);
int main(){
    FILE *fp1 = fopen("formatted_data.txt","r");
    FILE *fp2 = fopen("formatted_out.txt","w");
    if(fp1==NULL || fp2==NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];          int n = 5;
        read(a,n,fp1);
        int sum = find_sum(a,n);
        //fprintf(stdout,"sum is %d",sum);
        fprintf(fp2,"sum is %d",sum);
        fclose(fp1);  fclose(fp2);
    }
    return 0;
}
```



```
C:\Users\De11>formatted_out.txt
```

```
formatted_out - Notepad
File Edit Format View Help
sum is 110
```

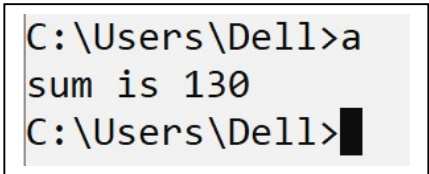
```
void read(int *a, int n, FILE *fp) {
    int i;
    for(i = 0; i < n; i++)
        { fscanf(fp, "%d", a+i); }
}
```

```
int find_sum(int *a, int n) {
    int i;
    int sum = 0;
    for(i = 0; i < n; i++)
        { sum = sum + *(a+i); }
    return sum;
}
```

Version 2: If we do not specify n in version 1, code fails. So use the return value of fscanf to find n

```
#include<stdio.h>
int read(int *a, FILE *fp);
int find_sum(int *a, int n);
int main(){
    FILE *fp1 = fopen("formatted_data.txt", "r");
    FILE *fp2 = fopen("formatted_out.txt", "w");
    if(fp1 == NULL || fp2 == NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];
        int n = read(a, fp1);
        int sum = find_sum(a, n); // same as version 1
        fprintf(stdout, "sum is %d", sum);
        fclose(fp1); fclose(fp2);
    }
    return 0;
}
```

```
int read(int *a, FILE *fp)
{
    int i = 0;
    while (fscanf(fp, "%d", a+i) != EOF)
        i++;
    return i;
}
```



```
C:\Users\De11>a
sum is 130
C:\Users\De11>
```

Block read / write operation on File

- **fwrite(const void *ptr, size_t size, size_t n, FILE *fp):** This function writes the data specified by the void pointer ptr to the file. **On success, it returns the count of the number of items successfully written to the file. On error, it returns a number less than n.**
- **fread(void *ptr, size_t size, size_t n, FILE *fp):** This function is commonly used to read binary data. It accepts the same arguments as the fwrite() function does. The function reads n items from the file where each item occupies the number of bytes specified in the second argument. **The function on success reads n items from the file and returns n and on error or end of the file, it returns a number less than n.**

Coding Example_6: Demo of fwrite and fread functions. Employee details to be read from the user and writing a data file.

```
#include<stdio.h>
#include<stdlib.h>
struct employee
{
    char name[15];    char designation[15];    int age;    float salary;
}emp;
int main()
{
    FILE *fp;
    fp = fopen("employee_data.txt", "w"); // at the end of the program execution, open and
    see the contents of the data file. Not in readable format
    if(fp == NULL)
        printf("Error opening file\n");
    else
    {
        char ch;
        do
        {
            printf("\nEnter the name:\t");
            scanf("%s",emp.name);
            printf("Enter the designation:\t");
            scanf("%s",emp.designation);
            printf("Enter the age : ");
            scanf("%d",&emp.age);
```

```
Enter the name: sindhu
Enter the designation: Asst.Professor
Enter the age : 30
Enter the salary : 9000000

Do you want to add another data (y/n) : y

Enter the name: indhu
Enter the designation: Asst.Prof
Enter the age : 80
Enter the salary : 8000000

Do you want to add another data (y/n) : n
```

NAME	DESIGNATION	AGE	SALARY
sindhu	Asst.Professor	30	9000000.000000
indhu	Asst.Prof	80	8000000.000000

```
        printf("Enter the salary : ");
        scanf("%f",&emp.salary);
        fwrite(&emp,sizeof(emp),1,fp);
        printf("\nDo you want to add another data (y/n) : ");
        fflush(stdin);
        ch = getchar();
    } while(ch=='y' || ch=='Y');
    fclose(fp);
    fp = fopen("employee_data.txt","r");
    if(fp == NULL)
        printf("cannot open the file\n");
    else
    {
        printf("\n\tNAME\tDESIGNATION\tAGE\tSALARY\n");
        while(fread(&emp,sizeof(emp),1,fp)>0)
            printf("\n\t%s\t%s\t%d\t%f\n",emp.name,emp.designation
            ,emp.age,emp.salary);
        fclose(fp);
    }
}
return 0;
}
```

Random Access to a file

You can jump to a specific position in the file and read/write data — without reading everything sequentially. Files permit non-sequential or random access to its contents using C functions. To access a file randomly, you open the file, seek a particular location, and read from or write to that file. In order to access a particular file or record C supports below functions.

fseek(), ftell(), rewind()

- **int fseek(FILE *stream, long offset, int whence);**

Used to set the file position indicator for the stream to a new position. This function accepts three arguments. The first argument is the FILE stream pointer returned by the fopen() function. The second argument ‘offset’ tells the amount of bytes to seek. The third argument ‘whence’ tells from where the seek of ‘offset’ number of bytes is to be done. The available values for whence are as below. Returns zero if successful, else it returns a non-zero value.

	Value	whence
SEEK_SET	0	Beginning of file
SEEK_CUR	1	Current position
SEEK_END	2	End of file

- **ftell(FILE *stream)**

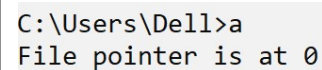
Returns the current file position of the given file_pointer. If an error occurs, -1 is returned.

- **rewind(FILE *stream)**

The function rewind sets the file_pointer position to the beginning of the file. Doesn't return any value.

Coding Example_7: Demo of ftell () . Pointer is at the beginning of the file just after opening the file in read mode

```
#include<stdio.h>
int main(){
    FILE *fp = fopen("data.txt","r"); // try w and a as well
    if(fp == NULL) printf("cannot open the file");
    else {
        printf("File pointer is at %d\n",ftell(fp));
        fclose(fp);
    }
    return 0;
}
```



```
C:\Users\Dell>a
File pointer is at 0
```

Coding Example_8: Demo of fseek()

```
#include<stdio.h>
int main()
{
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else
    {
        printf("Initially pointer is at %d\n",ftell(fp));
        fseek(fp,5,SEEK_SET); // start from the beginning
        printf("After fseek pointer is at %d\n",ftell(fp)); //5
        //fseek(fp,2,SEEK_SET);
        //printf("%d",ftell(fp)); // 2
    }
}
```

```

//fseek(fp,2,SEEK_CUR); //start from the current position of the pointer
//printf("%d",ftell(fp)); //7
fseek(fp,-9,SEEK_END); //start from the end of the file
printf("after seeking from end %d",ftell(fp));
printf("character just read is ");
putchar(fgetc(fp));
fclose(fp);
}
return 0;
}

```

```

C:\Users\Dell>a
Initially pointer is at 0
After fseek pointer is at 5
after seeking from end 22character just read is i

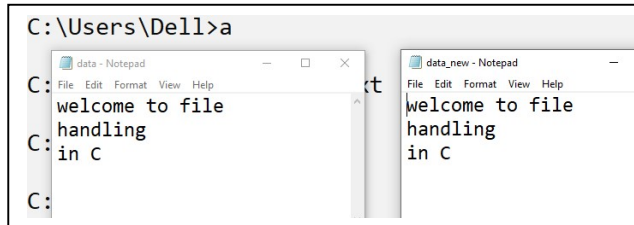
```

Coding Example_9: Copy the contents of one File to another

```

#include<stdio.h>
int main(){
    FILE *fp1 = fopen("data.txt","r");
    FILE *fp2 = fopen("data_new.txt","w");
    char ch;
    while((ch = fgetc(fp1)) != EOF)
        fputc(ch, fp2); // copy character by character from 5th position
    fclose(fp1); fclose(fp2);
    return 0;
}

```



Coding Example_10: Usage of rewind()

```

#include<stdio.h>
int main(){
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else{
        fseek(fp,-5,SEEK_END); printf("%d\n",ftell(fp));
        fseek(fp,0,SEEK_SET)//rewind(fp); to move the pointer to the beginning
of the file.
        printf("%d\n",ftell(fp));    fclose(fp);
        }return 0;
}

```

```

C:\Users\Dell>a
26
0

```

Handle Files, Handle Data – Happy Coding!